

Project Title: Resilience-Flow

Subtitle: Adaptive Protocol Pivoting and Reliable UDP for Congested Emergency Networks

1. Problem Statement & Motivation

During disasters (e.g., earthquakes), local ad-hoc or mesh networks set up by rescue teams often suffer from severe congestion. Even if unauthorized users cannot authenticate to the network, their devices generate massive "Probe Request" broadcast storms. This RF interference causes extreme CSMA/CA contention. Under such high packet loss (>30%) and high RTT conditions, standard TCP experiences severe head-of-line blocking and connection timeouts, rendering communication impossible.

2. Proposed Solution

Resilience-Flow proposes an **Adaptive Node Architecture**. The system operates on standard TCP during healthy network conditions but dynamically pivots to a highly optimized, custom Reliable-UDP layer the moment network metrics cross a critical failure threshold. This allows vital data to "pierce" through the noise without waiting for rigid TCP handshakes or timeouts.

3. Granular Implementation Steps

Phase 1: Real-Time Network Telemetry (The Monitor)

- Implement a background daemon that periodically exchanges lightweight "Heartbeat" packets (using Python struct for minimal byte size).
- Continuously calculate Round-Trip Time (RTT) and Packet Loss percentages using an exponential moving average.

Phase 2: Dynamic Protocol Pivot (State Machine)

- Define thresholds (e.g., LOSS > 30% or RTT > 500ms).
- Implement a 0ms-delay pivot mechanism: Once the threshold is crossed, the application immediately drops the TCP socket and routes all outgoing traffic through pre-allocated UDP sockets.

Phase 3: Custom Reliable UDP Layer (Workshop 4 Extension)

- **Header Compression:** Discard verbose JSON payloads used in standard workshops. Utilize Python's struct module to pack sequence numbers and message types into a minimal 5-byte header.
- **Selective-Repeat ARQ:** Implement the sliding-window and selective-repeat logic from **Workshop 4**. The receiver will individually ACK packets, and the sender will

precisely retransmit only the timed-out sequence numbers, ensuring reliability without the overhead of Go-Back-N or TCP congestion control.

Phase 4: Quality of Service (QoS) & Prioritization (Separation of Concerns)

- Implement an application-layer priority queue with distinct packet types to optimize bandwidth.
- **Explicit Text Prioritization:** Users will use the `/sos` command exclusively for critical emergency text messages (e.g., medical supply requests). These payloads bypass standard buffers and are injected immediately into the sliding window.
- **Event-Driven Telemetry:** Users can manually broadcast their GPS coordinates using the `/location` command.
- **The Priority Override:** When in CRISIS (UDP) mode, both `/sos` and `/location` packets are treated as highest priority, bypassing standard queues and triggering immediate, aggressive transmission.

4. Expected Deliverables & Demonstration

For the final submission, we will provide:

1. The fully documented Python implementation of the ResilienceNode.
2. A local simulation environment where artificial network congestion (packet loss and delay) can be toggled via CLI commands.
3. A demonstration showing two nodes gracefully degrading from standard TCP to the custom Survival UDP layer, successfully delivering SOS messages under simulated 40% packet loss.

Appendix: A Real-World Scenario: The "Golden Hours" of an Earthquake

1. The Setup (The Local Network): Imagine Istanbul, two hours after a major earthquake. Cellular towers and ISPs are completely down. In Kadıköy, a Rescue Command Center (Node A) and a frontline Medical Team (Node B) are located 500 meters apart. They establish a local, battery-powered Ad-Hoc Wi-Fi network to coordinate.

2. The Invisible Threat (RF Interference & Congestion): As desperate civilians and other rescue teams flood the disaster zone, a hidden problem emerges. Thousands of smartphones in the area are desperately scanning for connectivity. Even if they don't know the Wi-Fi password, these devices flood the airwaves with continuous "Probe Requests". This creates massive Radio Frequency (RF) interference and severe CSMA/CA contention. The wireless medium is literally choked with digital noise.

3. The Collapse of Standard Protocols: The medical team urgently needs supplies and uses a standard TCP-based application. However, because of the RF noise, packet

loss reaches 30%. Standard TCP interprets this as congestion, drastically reduces its window size, and fails to complete its 3-way handshake. The app freezes, displaying a "Connection Timeout" error. The medical team is cut off.

4. Resilience-Flow in Action: Here is how our project saves the communication line:

- **Detection:** The background Network Monitor on the medical team's laptop detects the anomaly: RTT jumps to 800ms and packet loss hits 40%.
- **The Pivot:** Instantly—with zero millisecond delay—the system executes the Dynamic Protocol Pivot, dropping the failing TCP stack and switching to "CRISIS" (Survival UDP) mode.
- **The S.O.S:** A medic types into the terminal: `/sos` Need tourniquets and hemostatic agents immediately!.
- **Piercing the Noise:** Instead of sending heavy, verbose TCP packets, Resilience-Flow wraps this critical text data in a minimalist compressed header. If the medic also needs to send coordinates, they can use `/location` to transmit a highly compressed GPS packet.
- **Guaranteed Delivery:** The airwaves are still chaotic, and some packets collide. However, our Selective-Repeat ARQ precisely and aggressively retransmits only the lost chunks within milliseconds.

5. The Impact: The message pierces through the broadcast storm and reaches the Command Center. Supplies are dispatched. Resilience-Flow doesn't miraculously fix a broken physical infrastructure, but it guarantees that when the network is dying, the most critical life-saving data still finds a way through.